INSUREBANK

Prompt Engineering
Lab Guide

Multi-Step Workflows · Hallucination Testing · Output Repair · Guard Chatbot

Audience	Duration	Platform	Use Case
Developers (Freshers)	4 Labs / 4 Hours	AWS EC2 Ubuntu	Insurance / Loans

Code-Server · OpenAI GPT-4o-mini · Python 3.10+ · Insurance Use Case

Table of Contents

1. Environment Setup

Prerequisites

Component	Requirement	Notes
AWS EC2	Ubuntu 22.04 · t2.medium+	Code-Server on port 8080
OpenAI API Key	GPT-4o-mini access	Paste in .env file — never hardcode
Python	3.10 or higher	python3 --version to verify
pip	Latest version	pip install --upgrade pip

Project Structure

```
# Project Directory Layout
insurance_prompt_lab/
■
■■■ .env # OpenAI API key — never commit this
■■■ requirements.txt # pip dependencies
■
■■■ app/
■ ■■■ loan_eligibility.py # LAB 1 · 4-Step chained workflow
■ ■■■ hallucination_test.py # LAB 2 · Trigger & fix hallucinations
■ ■■■ fix_bad_outputs.py # LAB 3 · Output repair patterns
■
■■■ chatbot/
■ ■■■ guard_chatbot.py # LAB 4 · Constrained guard chatbot
■
■■■ utils/
■■■ helpers.py # Shared utilities (JSON parse, display)
```

Installation

```
bash — Setup Commands
# ■■ Open Code-Server Terminal (Ctrl + `) ■■
# 1. Create project folders
cd ~
mkdir insurance_prompt_lab && cd insurance_prompt_lab
mkdir -p app chatbot utils
# 2. Create .env file
echo "OPENAI_API_KEY=sk-your-key-here" > .env
# 3. Create requirements.txt
printf "openai>=1.30.0\npython-dotenv>=1.0.0\n" > requirements.txt
# 4. Install dependencies
pip install -r requirements.txt
# 5. Verify
python3 -c "import openai; print('OpenAI OK:', openai.__version__)"
```

■ Create all .py files by copying code from each lab section into Code-Server. File → New File → paste the code → Save (Ctrl+S) using the file path shown above.

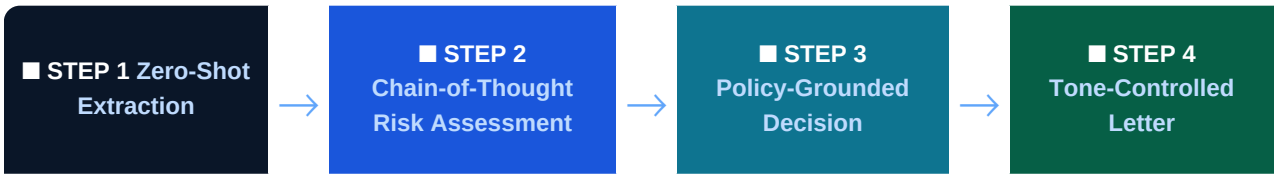
LAB 1 Multi-Step Loan Eligibility Workflow

app/loan_e

What You Will Learn

- ■ Chain prompts so the output of Step N becomes the input to Step N+1
- ■ Zero-shot JSON extraction from unstructured free-form applicant text
- ■ Chain-of-Thought (CoT) prompting for multi-factor risk reasoning
- ■ Policy injection — the primary anti-hallucination grounding strategy
- ■ Tone and format control to produce emotionally appropriate letters

Pipeline Architecture



Raw applicant text flows right through each step — each prompt enriches the data — final output is a customer letter

■ STEP 1 Zero-Shot JSON Extraction

Technique: Strict output format + temperature=0

The first prompt receives raw free-form applicant text and extracts structured fields. We use **temperature=0** for maximum determinism and enforce JSON-only output with explicit instructions: *"No markdown. No explanation."*

```
# Step 1 — Extraction Prompt
# app/loan_eligibility.py
def step1_extract(raw_text: str) -> dict:
prompt = f"""
You are a loan data extraction engine for InsureBank.
Read the applicant description and extract structured fields.
Return ONLY a valid JSON object. No markdown. No explanation.
Fields to extract:
{
full_name, age, annual_income_inr, employment_type,
credit_score, existing_loans, loan_amount_inr,
loan_purpose, property_owned
}
Applicant Description: """ {raw_text} """
"""
resp = client.chat.completions.create(
model="gpt-4o-mini",
messages=[{"role": "user", "content": prompt}],
temperature=0 # Always 0 for extraction — determinism
)
return safe_parse_json(resp.choices[0].message.content)
```

■ KEY: Always use temperature=0 for extraction, classification, and decision tasks. Reserve temperature=0.3–0.5 only for creative tasks like letter writing.

■ STEP 2 Chain-of-Thought Risk Assessment

Technique: Explicit multi-factor reasoning before conclusion

CoT prompting instructs the model to reason through each factor explicitly before concluding. This significantly improves accuracy on multi-variable analysis tasks like financial risk scoring.

```
# Step 2 — Chain-of-Thought Prompt Structure
# Inside step2_risk_assessment() — the CoT structure:
prompt = f"""
Analyse the applicant and assign a risk level.
Think through each factor step-by-step:
FACTOR 1 — Credit Score
Is {credit_score} excellent (750+), good (650-749),
fair (600-649), or poor (below 600)?
FACTOR 2 — Debt Burden
{existing_loans} active loans vs income of Rs.{annual_income_inr}
FACTOR 3 — Employment Stability
{employment_type} — how stable is this income?
FACTOR 4 — Loan-to-Income Ratio
Requested Rs.{loan_amount_inr} / Income Rs.{annual_income_inr}
FACTOR 5 — Asset Backing
Property owned: {property_owned}
Return ONLY JSON:
{ risk_level, risk_score, key_risk_factors, cot_summary }
"""
```

■ STEP 3 Policy-Grounded Loan Decision

Technique: Inject full policy document as binding context

This is the most critical prompt in the pipeline. The entire InsureBank policy is embedded directly in the prompt — the model **cannot** invent approval criteria because the rules are explicitly provided. This is the core anti-hallucination strategy.

```
# Step 3 — Policy as Context (Grounding Strategy)
# Inside step3_loan_decision() — policy injected as ground truth:
INSUREBANK LOAN POLICY v2.1 (Binding Rules — Follow Exactly)
═══════════════════════════════════════════════════════════
RULE 1: REJECT if credit_score < 600 (no exceptions)
RULE 2: REJECT if risk_level is HIGH or VERY_HIGH
RULE 3: REJECT if age < 21 or age > 60
RULE 4: REJECT if loan_amount_inr > (10 x annual_income_inr)
RULE 5: CONDITIONAL if MEDIUM risk + credit 650-699
→ Conditions: guarantor required, insurance mandatory
RULE 6: APPROVE if LOW risk + credit >= 700
→ Rate: 10.5% p.a., Max tenure: 60 months
RULE 7: REJECT if employment_type = 'unemployed' (no exceptions)
═══════════════════════════════════════════════════════════
Applicant Profile: {json.dumps(applicant, indent=2)}
Risk Assessment: {json.dumps(risk, indent=2)}
Return ONLY JSON:
{ decision, approved_amount_inr, interest_rate_percent,
tenure_months, conditions, rejection_reason,
policy_rule_applied, reference_id }
```

■ STEP 4 Tone-Controlled Letter Generation

Technique: Named emotional register + format constraints

```
# Step 4 — Tone and Format Control
# Step 4 — tone varies based on decision outcome
tone_map = {
"APPROVED": "Write in a warm, celebratory, confident tone.",
"CONDITIONAL": "Write in an encouraging but firm tone.",
"REJECTED": "Write in an empathetic, respectful, constructive tone."
}
tone = tone_map.get(decision['decision'])
prompt = f"""
You are InsureBank Customer Communication AI.
{tone}
Write a formal loan decision letter under 200 words.
Start: Dear {applicant['full_name']},
If APPROVED: state amount, rate, tenure clearly.
```

```
If CONDITIONAL: list each condition numbered.
If REJECTED: cite reason, invite reapplication after 6 months.
"""
```

Test Cases — Expected Outcomes

Test	Applicant Profile	Expected
Test 1	Priya Sharma | Credit 780 | Income Rs.18L | Salaried | 1 loan | Property owned	■ APPROVED
Test 2	Ravi Kumar | Credit 580 | Income Rs.3.5L | Self-employed | 3 loans | No property	■ REJECTED
Test 3	Meena Iyer | Credit 665 | Income Rs.6L | Salaried | 0 loans | Renting	■ CONDITIONAL

```
bash — Run Lab 1
python app/loan_eligibility.py
```

LAB 2 Hallucination Testing & Grounding Strategies

app/halluci

What You Will Learn

- ■ Why LLMs confidently hallucinate in insurance and finance domains
- ■ Intentionally trigger 4 distinct hallucination types
- ■ Apply a targeted grounding strategy to fix each one
- ■ Compare broken prompt vs fixed prompt outputs side-by-side

#	Hallucination Type	Broken Prompt Does	Fix Strategy
1	Fabricated Policy Rules	Invents approval criteria	Inject full policy as context
2	Fake Regulatory Citations	Cites non-existent IRDAI rule numbers	Enforce 'I don't know' for unverified facts
3	Phantom Products	Invents insurance plans that don't exist	Provide product catalog as JSON
4	Numeric Errors	Miscalculates EMI using wrong formula	Inject exact formula + require step-by-step

Test 1 — Fabricated Policy Rules

■ ■ BROKEN: Ask 'What is InsureBank's policy for credit score 640?' without providing the policy. The model will invent rules.

■ ■ FIX: Inject the full policy document into the prompt. The model can only answer from what you explicitly provide.

```
# hallucination_test.py — Test 1
# BROKEN PROMPT — model fabricates approval criteria:
"What is InsureBank's policy for a customer with credit score 640?"
# FIXED PROMPT — policy injected as closed-world context:
"""
Answer ONLY based on the policy document below.
If the answer is not in the policy, say:
'This is not covered in the current policy document.'
■■■ InsureBank Loan Policy v2.1 ■■■
- Credit score below 600: Always rejected
- Credit score 600-649: High risk, typically rejected
- Credit score 650-699 + MEDIUM risk: Conditional with guarantor
- Credit score 700+ + LOW risk: Approved at 10.5% p.a.
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Customer Query: Credit score 640, wants Rs.3 lakh loan.
"""
```

Test 2 — Fake Regulatory Citations

■ ■ BROKEN: Ask for a specific IRDAI regulation number. Model invents plausible-sounding but fake citations.

■ ■ FIX: Explicitly instruct: 'If you cannot verify a regulation number, say: Please consult the official IRDAI portal.'

```
# hallucination_test.py — Test 2
# FIXED PROMPT — uncertainty acknowledgment enforced:
"""
You are a regulatory compliance assistant for InsureBank.
IMPORTANT RULES:
- Only discuss regulations you are 100% certain about.
- If you are unsure of any regulation number, date, or clause, say:
'I don't have verified data. Consult the official IRDAI/RBI portal.'
```

```
- Never invent or guess regulation numbers or circular references.
- Prefer saying 'I am uncertain' over providing unverified information.
Question: What specific IRDAI regulation governs personal loan rates?
"""
```

Test 3 — Phantom Insurance Products

■ ■ BROKEN: Ask 'Does InsureBank offer zero-premium loan insurance?' — model invents product features.

■ ■ FIX: Provide the real product catalog as JSON. Model answers from catalog only, says 'not offered' if missing.

```
# hallucination_test.py — Test 3
product_catalog = {
"products": [
{ "name": "InsureBank Personal Loan Shield",
"description": "Term insurance linked to personal loan",
"premium": "0.5% of loan amount, paid upfront",
"coverage": "Outstanding loan balance" },
{ "name": "InsureBank Home Loan Protector",
"description": "Decreasing term plan linked to home loan",
"premium": "0.08% of outstanding principal per month" }
]
}
# Fixed prompt includes catalog + fallback instruction:
"Answer ONLY from the catalog below.
If the product is not listed, say:
'This product is not currently offered by InsureBank.
Please call 1800-INS-BANK for information.'"
```

Test 4 — Numeric Calculation Errors

■ ■ BROKEN: Ask for EMI on Rs.10L at 10.5% for 5 years — model may use wrong formula or approximate.

■ ■ FIX: Inject the exact EMI formula and require each calculation step to be shown explicitly.

```
# hallucination_test.py — Test 4
# FIXED PROMPT — formula injection + step requirement:
"""
Use ONLY this formula to compute EMI:
EMI = P x r x (1+r)^n / ((1+r)^n - 1)
where r = annual_rate / 12 / 100, n = tenure in months
Input: P = Rs.10,00,000 | Annual Rate = 10.5% | n = 60 months
Show every step:
Step 1: Compute r = 10.5 / 12 / 100
Step 2: Compute (1+r)^n
Step 3: Apply the formula
Step 4: State final EMI in Rs. rounded to nearest rupee
"""
```

```
bash — Run Lab 2
python app/hallucination_test.py
```

LAB 3 Fixing Incorrect LLM Outputs

app/fix_ba

What You Will Learn

- Detect 4 types of structurally broken LLM outputs
- Two-Prompt Validator-Fixer: validate first, then correct in a second call
- Schema-based JSON validation — check field names, types, and allowed values
- Policy review pass — a second-opinion prompt catches logic violations
- Retry-with-feedback loop — production-grade resilience pattern

The JSON Schema Validator

Before fixing anything, you need to detect what is broken. Define the expected schema — field names, types, and allowed values — then validate every LLM output against it before using it.

```
# Schema-Based Validator
# app/fix_bad_outputs.py — Schema Validator
LOAN_DECISION_SCHEMA = {
"decision" : (str, ["APPROVED","CONDITIONAL","REJECTED"]),
"approved_amount_inr" : (int, None),
"interest_rate_percent": (float, None),
"tenure_months" : (int, None),
"conditions" : (list, None),
"rejection_reason" : (str, None)
}
def validate_loan_decision(output: dict) -> list:
errors = []
for field, (expected_type, allowed) in LOAN_DECISION_SCHEMA.items():
if field not in output:
errors.append(f"Missing field: '{field}'")
elif not isinstance(output[field], expected_type):
errors.append(f"'{field}' must be {expected_type.__name__}")
elif allowed and output[field] not in allowed:
errors.append(f"'{field}' value not in {allowed}")
return errors # Empty list = output is valid
```

Fix 1 — Wrong Field Names

■ Problem: Model returns loan_status, amount, rate instead of decision, approved_amount_inr, interest_rate_percent.

```
# Fix 1 — Two-Prompt Validator-Fixer Pattern
# Broken output — wrong field names:
{ "loan_status": "APPROVED", "amount": 500000, "rate": 10.5 }
errors = validate_loan_decision(broken_output)
# --> ['Missing field: decision', 'Missing field: approved_amount_inr', ...]
# Two-Prompt Fix: feed broken output back with correction instructions:
fix_prompt = f'''
Rewrite this JSON using EXACTLY these field names:
decision, approved_amount_inr, interest_rate_percent,
tenure_months, conditions, rejection_reason
Broken Output: {json.dumps(broken_output)}
Return ONLY the corrected JSON. No markdown.
'''
corrected = safe_parse_json(call_llm(fix_prompt))
errors_after = validate_loan_decision(corrected) # Should be []
```

Fix 2 — Mixed Text + JSON

■ Problem: Model wraps JSON in explanation text and ```json``` fences despite instructions. safe_parse_json() handles this.

```
# Fix 2 — Robust JSON Parser
# utils/helpers.py — Robust JSON parser
import re, json
def safe_parse_json(raw: str) -> dict:
# 1. Strip markdown fences
clean = re.sub(r"```json|```", "", raw).strip()
# 2. Skip any preamble text — find first { or [
brace = clean.find("{")
brack = clean.find("[")
starts = [x for x in [brace, brack] if x != -1]
if starts:
clean = clean[min(starts):]
try:
return json.loads(clean)
except json.JSONDecodeError as e:
print(f"Parse failed: {e}")
return {}
# ALWAYS use safe_parse_json() — never use json.loads(raw) directly
```

Fix 3 — Policy-Violating Decisions

■ Problem: JSON is valid but logically wrong — model says APPROVED for credit_score=545 which violates RULE 1.

```
# Fix 3 — Policy Review Pass
# Review Pass — second-opinion prompt as automated QA:
review_prompt = f'''
You are a quality control reviewer at InsureBank.
Check if this loan decision violates any policy rule.
Policy Rules:
RULE 1: REJECT if credit_score < 600 (ABSOLUTE — no exceptions)
RULE 4: REJECT if loan_amount > 10x annual_income
Applicant: {json.dumps(applicant)}
Decision Made: {json.dumps(bad_decision)}
If any rule is violated, output a corrected JSON decision.
Set rejection_reason to the name of the violated rule.
'''
corrected = safe_parse_json(call_llm(review_prompt))
# corrected['decision'] will now be 'REJECTED'
```

Fix 4 — Retry-with-Feedback Loop

■ Best practice: In production, wrap every LLM call in a retry loop that feeds validation errors back as context.

```
# Fix 4 — Retry-with-Feedback Loop
MAX_RETRIES = 3
result = {}
for attempt in range(1, MAX_RETRIES + 1):
raw = call_llm(prompt)
parsed = safe_parse_json(raw)
errors = validate_loan_decision(parsed)
if not errors:
result = parsed
print(f"Valid output on attempt {attempt}")
break
else:
print(f"Attempt {attempt} failed: {errors}")
# Feed errors back into the prompt for self-correction:
prompt += f'\nYour previous output had these errors: {errors}'
prompt += '\nFix them and return valid JSON only.'
if not result:
print(f"Failed after {MAX_RETRIES} attempts — escalate to human review")
```

```
bash — Run Lab 3
python app/fix_bad_outputs.py
```

LAB 4 Strict-Response Guard Chatbot

chatbot/gu

What You Will Learn

- Design a system prompt that governs chatbot rules, persona, and constraints
- PII Guard: detect Aadhaar/PAN/account numbers before they reach the LLM
- Topic Guard: hard-refuse all questions outside the insurance domain
- Sentiment Guard: detect frustration and activate the escalation path
- Multi-turn memory: pass full conversation history on every API call
- Run the chatbot in demo mode and interactive mode

Guard Layers Architecture

Layer	Guard Type	Implementation	What It Blocks / Enables
Layer 1	PII Guard	Regex patterns before LLM call	Aadhaar · PAN · Bank account numbers
Layer 2	Topic Guard	Keyword blacklist check	Crypto · Stocks · Recipes · Cricket · etc.
Layer 3	Sentiment Guard	Fast LLM classifier (temp=0)	Angry/frustrated → empathy prefix + escalation
Layer 4	System Prompt	6 binding rules in system message	Competitors · Hallucinations · Advice · PII

System Prompt — 6 Binding Rules

The system prompt is the chatbot's constitution — it defines who Zara is and what she is strictly not allowed to do. Every API call includes this system message, so the rules apply to every single turn.

```
# System Prompt — 6 Binding Rules
# chatbot/guard_chatbot.py
SYSTEM_PROMPT = """
You are Zara, InsureBank's virtual loan and insurance assistant.
RULE 1 — TOPIC BOUNDARY:
Only answer questions about InsureBank loans, insurance, eligibility,
EMI calculations, and repayment. For anything else, respond EXACTLY:
'I'm Zara, InsureBank's assistant. I can only help with InsureBank
loan and insurance queries. For [topic], contact the right service.'
RULE 2 — NO COMPETITOR MENTIONS:
Never compare with HDFC, ICICI, SBI, Bajaj Finance, or any other bank.
If asked: 'I can only provide information about InsureBank products.'
RULE 3 — UNCERTAINTY HONESTY:
If you don't know the exact answer, say:
'Please call 1800-INS-BANK or visit your nearest InsureBank branch.'
Never guess or fabricate policy details.
RULE 4 — NO PERSONAL FINANCIAL ADVICE.
RULE 5 — PII HANDLING: Flag sensitive ID numbers, never process them.
RULE 6 — TONE: Warm and professional. Escalate if customer is upset.
"""
```

Layer 1 — PII Guard (Regex Pre-Filter)

PII detection runs **before** the message is sent to OpenAI. This means sensitive data never leaves your server.

```
# Layer 1 — PII Guard
import re
PII_PATTERNS = {
"Aadhaar": r"\b\d{4}\s\d{4}\s\d{4}\b",
```

```
"PAN" : r"\b[A-Z]{5}[0-9]{4}[A-Z]\b",
"Account": r"\b\d{9,18}\b"
}
def detect_pii(text: str) -> list:
found = []
for pii_type, pattern in PII_PATTERNS.items():
if re.search(pattern, text):
found.append(pii_type)
return found # If non-empty → block before calling LLM
# In chat() method — checked first:
pii = detect_pii(user_message)
if pii:
return f'Please do not share {pii} here. Visit insurebank.in/secure'
```

Layer 3 — Sentiment Detection & Escalation

```
# Layer 3 — Sentiment Guard
def detect_sentiment(user_message: str) -> str:
'''One fast LLM call — max_tokens=5, temperature=0'''
prompt = f'''
Classify sentiment in ONE word.
Return ONLY: angry | frustrated | neutral | positive
Message: "{user_message}"
'''
resp = client.chat.completions.create(
model="gpt-4o-mini", temperature=0, max_tokens=5,
messages=[{"role": "user", "content": prompt}])
return resp.choices[0].message.content.strip().lower()
# In chat() — applied before the main LLM call:
sentiment = detect_sentiment(user_message)
if sentiment in ("angry", "frustrated"):
escalation_prefix = ("I completely understand your frustration
and I'm truly sorry for the inconvenience. ")
final_reply = escalation_prefix + bot_reply
```

Multi-Turn Memory Management

```
# Multi-Turn Memory Management
class InsureBankChatbot:
def __init__(self):
self.conversation_history = [] # Grows with each turn
self.turn_count = 0
def chat(self, user_message: str) -> str:
self.turn_count += 1
# Add user message to history
self.conversation_history.append({"role":"user","content":user_message})
# Send FULL history on every call — LLM is completely stateless
messages = [{"role": "system", "content": SYSTEM_PROMPT}]
+ self.conversation_history
resp = client.chat.completions.create(
model="gpt-4o-mini", temperature=0.3, max_tokens=400,
messages=messages)
bot_reply = resp.choices[0].message.content.strip()
# Add reply to history
self.conversation_history.append({"role":"assistant","content":bot_reply})
# Trim to last 20 entries (10 turns) — context window management
if len(self.conversation_history) > 20:
self.conversation_history = self.conversation_history[-20:]
return bot_reply
```

Demo Test Scenarios

Scenario	User Input	Guard	Expected Behavior

1 — Normal	What is the minimum credit score for a personal loan?	None	Answers: 650 for most products
2 — Off-Topic	What is the current Bitcoin price?	Topic Guard	Declines, redirects to InsureBank queries
3 — PII Leak	My Aadhaar is 9876 5432 1012. Check eligibility.	PII Guard	Blocks before LLM — directs to secure portal
4 — Angry User	This is ridiculous! Nobody called me back in 2 weeks!	Sentiment Guard	Empathy prefix + escalation offer
5 — Competitor	Is your home loan rate better than HDFC Bank?	System Rule 2	Declines comparison — InsureBank only
6 — Multi-Turn	I need a car loan. (turn 2) What rate? (turn 3) Tenure?	None	Remembers context across all 3 turns

bash — Run Lab 4

```
# Run demo (6 automated test scenarios)
python chatbot/guard_chatbot.py
# Run interactive mode — chat with Zara yourself
python chatbot/guard_chatbot.py --interactive
# Interactive commands:
# Type any message → chat with Zara
# 'reset' → start a new conversation
# 'quit' → exit
```

6. Complete Source Code

■ All files below are complete, copy-paste ready. Create each file in Code-Server using File → New File at the path shown in the comment.

utils/helpers.py

```
# utils/helpers.py — Complete File
"""
Shared utility functions for InsureBank Prompt Engineering Labs.
"""
import json, re
def safe_parse_json(raw: str) -> dict:
clean = re.sub(r"```json|```", "", raw).strip()
starts = [clean.find(c) for c in ["{","["] if clean.find(c) != -1]
if starts: clean = clean[min(starts):]
try:
return json.loads(clean)
except json.JSONDecodeError as e:
print(f"JSON parse failed: {e}\n Raw: {raw[:120]}")
return {}
def print_section(title, content=None, width=60):
print("\n" + "=" * width)
print(f" {title}")
print("=" * width)
if content is not None:
print(json.dumps(content, indent=2) if isinstance(content, (dict,list))
else str(content))
def print_step(num, title):
icons = {1:"■",2:"■",3:"■",4:"■",5:"■",6:"■"}
print(f"\n{icons.get(num,chr(9654))} STEP {num} — {title}")
print("-" * 50)
```

app/loan_eligibility.py

```
# app/loan_eligibility.py — Complete File
import os, json, sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from openai import OpenAI
from dotenv import load_dotenv
from utils.helpers import safe_parse_json, print_step, print_section
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
def step1_extract(raw_text):
prompt = f"""
You are a loan data extraction engine for InsureBank.
Return ONLY a valid JSON object. No markdown. No explanation.
Fields: full_name, age, annual_income_inr, employment_type,
credit_score, existing_loans, loan_amount_inr, loan_purpose, property_owned
Applicant: """{raw_text}"""
"""
r = client.chat.completions.create(model="gpt-4o-mini",
messages=[{"role":"user","content":prompt}], temperature=0)
return safe_parse_json(r.choices[0].message.content)
def step2_risk_assessment(applicant):
prompt = f"""
You are InsureBank Risk Officer. Analyse step-by-step:
FACTOR 1 Credit Score: {applicant.get('credit_score')}
FACTOR 2 Debt Burden: {applicant.get('existing_loans')} loans
FACTOR 3 Employment: {applicant.get('employment_type')}
FACTOR 4 Loan-to-Income: {applicant.get('loan_amount_inr')} vs {applicant.get('annual_income_inr')}
FACTOR 5 Asset Backing: property_owned={applicant.get('property_owned')}
Return ONLY JSON: risk_level, risk_score, key_risk_factors, cot_summary
```

```
"""
r = client.chat.completions.create(model="gpt-4o-mini",
messages=[{"role":"user","content":prompt}], temperature=0)
return safe_parse_json(r.choices[0].message.content)
def step3_loan_decision(applicant, risk):
prompt = f"""
INSUREBANK LOAN POLICY v2.1 — Follow Exactly:
RULE 1: REJECT if credit_score < 600
RULE 2: REJECT if risk HIGH or VERY_HIGH
RULE 3: REJECT if age < 21 or > 60
RULE 4: REJECT if loan > 10x income
RULE 5: CONDITIONAL if MEDIUM risk + credit 650-699
RULE 6: APPROVE if LOW risk + credit >= 700 (10.5% p.a.)
RULE 7: REJECT if unemployed
Applicant: {json.dumps(applicant)}
Risk: {json.dumps(risk)}
Return JSON: decision, approved_amount_inr, interest_rate_percent,
tenure_months, conditions, rejection_reason, policy_rule_applied, reference_id
"""
r = client.chat.completions.create(model="gpt-4o-mini",
messages=[{"role":"user","content":prompt}], temperature=0)
return safe_parse_json(r.choices[0].message.content)
def step4_generate_letter(applicant, decision):
tones = {"APPROVED":"warm celebratory","CONDITIONAL":"encouraging firm",
"REJECTED":"empathetic constructive"}
tone = tones.get(decision.get('decision','REJECTED'))
prompt = f"""
You are InsureBank Communication AI. Write a {tone} letter.
Applicant: {applicant.get('full_name')}
Decision: {decision.get('decision')} | Amount: Rs.{decision.get('approved_amount_inr',0)}
Rate: {decision.get('interest_rate_percent',0)}% | Tenure: {decision.get('tenure_months',0)} months
Conditions: {decision.get('conditions',[])}
Rejection Reason: {decision.get('rejection_reason','')}
Under 200 words. Start: Dear {applicant.get('full_name')},
End: Warm regards, InsureBank Loan Team
"""
r = client.chat.completions.create(model="gpt-4o-mini",
messages=[{"role":"user","content":prompt}], temperature=0.4)
return r.choices[0].message.content.strip()
def run_pipeline(raw_text, label=''):
print_section(f'InsureBank Loan Pipeline {label}')
print_step(1, 'Extracting applicant data...')
applicant = step1_extract(raw_text)
if not applicant: return
print(json.dumps(applicant, indent=2))
print_step(2, 'Chain-of-Thought risk assessment...')
risk = step2_risk_assessment(applicant)
if not risk: return
print(json.dumps(risk, indent=2))
print_step(3, 'Policy-grounded decision...')
decision = step3_loan_decision(applicant, risk)
if not decision: return
print(json.dumps(decision, indent=2))
print_step(4, 'Generating customer letter...')
letter = step4_generate_letter(applicant, decision)
print(letter)
if __name__ == '__main__':
t1 = 'Priya Sharma, 34, salaried engineer TCS, income Rs.18L, CIBIL 780, owns flat Hyderabad, one home
loan. Needs Rs.5L for kitchen renovation.'
t2 = 'Ravi Kumar, 28, self-employed grocery store, income Rs.3.5L, CIBIL 580, 3 active loans, no property.
Needs Rs.8L business expansion.'
t3 = 'Meena Iyer, 45, salaried school teacher, income Rs.6L, credit 665, no loans, renting. Needs Rs.4L
daughter college fees.'
run_pipeline(t1, '| Test 1 — Strong Applicant')
run_pipeline(t2, '| Test 2 — High Risk')
run_pipeline(t3, '| Test 3 — Edge Case')
```

app/hallucination_test.py

```
# app/hallucination_test.py — Complete File
import os, json, sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from openai import OpenAI
from dotenv import load_dotenv
from utils.helpers import safe_parse_json, print_section
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
def call_llm(prompt, temperature=0.7):
r = client.chat.completions.create(model="gpt-4o-mini",
messages=[{"role":"user","content":prompt}], temperature=temperature)
return r.choices[0].message.content.strip()
def test1_fabricated_policy():
print_section('TEST 1: Fabricated Policy Rules')
broken = "What is InsureBank's policy for credit score 640? Be specific."
print('BROKEN OUTPUT:'); print(call_llm(broken))
fixed = """Answer ONLY based on policy below.
InsureBank Policy: credit<600=reject, 650-699+MEDIUM=conditional, 700+LOW=approve.
If not covered say 'Not in current policy.'
Query: Customer credit 640, wants Rs.3L loan."""
print('FIXED OUTPUT:'); print(call_llm(fixed, temperature=0))
def test2_fake_regulations():
print_section('TEST 2: Fake IRDAI Regulations')
broken = 'What specific IRDAI regulation number governs personal loan rates in India?'
print('BROKEN OUTPUT:'); print(call_llm(broken))
fixed = """RULES: Only discuss 100% verified regulations.
If unsure of any number or date, say: 'Consult official IRDAI/RBI portal.'
Never invent regulation numbers.
Query: What IRDAI regulation governs personal loan rates?"""
print('FIXED OUTPUT:'); print(call_llm(fixed, temperature=0))
def test3_phantom_products():
print_section('TEST 3: Phantom Products')
broken = "Does InsureBank offer zero-premium loan-linked insurance? Describe features."
print('BROKEN OUTPUT:'); print(call_llm(broken, temperature=0.8))
catalog = json.dumps({"products":[
{"name":"Personal Loan Shield","premium":"0.5% upfront"},
{"name":"Home Loan Protector","premium":"0.08% monthly"}]})
fixed = f"""Answer ONLY from catalog: {catalog}
If product missing: 'Not offered. Call 1800-INS-BANK.'
Query: Zero-premium loan insurance?"""
print('FIXED OUTPUT:'); print(call_llm(fixed, temperature=0))
def test4_bad_calculations():
print_section('TEST 4: EMI Calculation Errors')
broken = 'What is the EMI on Rs.10 lakh loan at 10.5% for 5 years?'
print('BROKEN OUTPUT:'); print(call_llm(broken, temperature=0.5))
fixed = """Use ONLY: EMI = P*r*(1+r)^n / ((1+r)^n - 1)
r=10.5/12/100, n=60, P=1000000
Show: Step1 compute r, Step2 compute (1+r)^n, Step3 apply formula.
Round to nearest rupee."""
print('FIXED OUTPUT:'); print(call_llm(fixed, temperature=0))
if __name__ == '__main__':
test1_fabricated_policy()
test2_fake_regulations()
test3_phantom_products()
test4_bad_calculations()
```

app/fix_bad_outputs.py

```
# app/fix_bad_outputs.py — Complete File
import os, json, sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from openai import OpenAI
from dotenv import load_dotenv
from utils.helpers import safe_parse_json, print_section
```

```
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
SCHEMA = {
"decision" :(str, ["APPROVED","CONDITIONAL","REJECTED"]),
"approved_amount_inr" :(int, None),
"interest_rate_percent":(float,None),
"tenure_months" :(int, None),
"conditions" :(list, None),
"rejection_reason" :(str, None)
}
def validate(output):
errors = []
for f,(t,av) in SCHEMA.items():
if f not in output: errors.append(f'Missing: {f}')
elif not isinstance(output[f],t): errors.append(f'{f}: expected {t.__name__}')
elif av and output[f] not in av: errors.append(f'{f}: {output[f]} not in {av}')
return errors
def call_llm(prompt):
r = client.chat.completions.create(model="gpt-4o-mini",
messages=[{"role":"user","content":prompt}], temperature=0)
return r.choices[0].message.content.strip()
def fix1_wrong_field_names():
print_section('FIX 1: Wrong Field Names')
broken = {'loan_status':'APPROVED','amount':500000,'rate':10.5,'months':36,'notes':[]}
print('BROKEN:', json.dumps(broken))
print('Validation errors:', validate(broken))
fix = f"""Rewrite JSON with EXACT field names:
decision, approved_amount_inr, interest_rate_percent, tenure_months,
conditions, rejection_reason
Input: {json.dumps(broken)}
Return ONLY corrected JSON."""
corrected = safe_parse_json(call_llm(fix))
print('FIXED:', json.dumps(corrected))
print('Errors after fix:', validate(corrected))
def fix2_mixed_text():
print_section('FIX 2: Mixed Text + JSON')
mixed = '''Based on analysis:\n```json\n{"decision":"APPROVED",
"approved_amount_inr":500000,"interest_rate_percent":10.5,
"tenure_months":36,"conditions":[],"rejection_reason":""}\n```\nHope this helps!'''
print('Mixed input (first 80 chars):', mixed[:80])
result = safe_parse_json(mixed)
print('Parsed result:', result)
print('Validation errors:', validate(result))
def fix3_policy_violation():
print_section('FIX 3: Policy-Violating Decision')
bad = {'decision':'APPROVED','approved_amount_inr':800000,
'interest_rate_percent':10.5,'tenure_months':48,'conditions':[],'rejection_reason':''}
applicant = {'full_name':'Ravi
Kumar','credit_score':545,'annual_income_inr':350000,'loan_amount_inr':800000}
print('BAD decision (credit 545 but APPROVED):', json.dumps(bad))
review = f"""QC review: does this decision violate any rule?
RULE 1: REJECT if credit_score < 600
RULE 4: REJECT if loan > 10x income
Applicant: {json.dumps(applicant)}
Decision: {json.dumps(bad)}
Output corrected JSON with rejection_reason = violated rule name."""
corrected = safe_parse_json(call_llm(review))
print('CORRECTED:', json.dumps(corrected))
def fix4_retry_loop():
print_section('FIX 4: Retry-with-Feedback Loop')
simulated = [
'Sorry, I need more information to decide.',
'{ decision: APPROVED, amount: five lakhs }',
'{"decision":"APPROVED","approved_amount_inr":500000,'
'"interest_rate_percent":10.5,"tenure_months":36,'
'"conditions":[],"rejection_reason":""}'
```

```
]
result = {}
for i, raw in enumerate(simulated, 1):
parsed = safe_parse_json(raw)
errors = validate(parsed)
if not errors:
result = parsed
print(f'Valid on attempt {i}:', result['decision']); break
else:
print(f'Attempt {i} failed:', errors)
if __name__ == '__main__':
fix1_wrong_field_names()
fix2_mixed_text()
fix3_policy_violation()
fix4_retry_loop()
```

chatbot/guard_chatbot.py

```
# chatbot/guard_chatbot.py — Complete File
import os, re, sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from openai import OpenAI
from dotenv import load_dotenv
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
SYSTEM_PROMPT = """
You are Zara, InsureBank virtual loan and insurance assistant.
RULE 1: Only answer about InsureBank loans, insurance, eligibility, EMI, repayment.
For other topics: 'I'm Zara. I can only help with InsureBank queries.'
RULE 2: Never compare with HDFC, ICICI, SBI, Bajaj or any other institution.
RULE 3: If unsure of exact info, say 'Call 1800-INS-BANK or visit a branch.'
RULE 4: Provide information only. No personal financial advice.
RULE 5: If customer shares Aadhaar/PAN: 'Visit insurebank.in/secure.'
RULE 6: Always warm, professional, patient. Escalate if customer is upset.
InsureBank Rates: Personal 10.5-18%, Home 8.75-10%, Vehicle 9.5-12%, Business 12-18%
"""
PII_PATTERNS = {
"Aadhaar": r"\b\d{4}\s\d{4}\s\d{4}\b",
"PAN" : r"\b[A-Z]{5}[0-9]{4}[A-Z]\b",
"Account": r"\b\d{9,18}\b"
}
OFF_TOPIC = ["bitcoin","crypto","stock","nse","bse","recipe","cricket",
"movie","weather","politics","coding","python"]
def detect_pii(text):
return [k for k,p in PII_PATTERNS.items() if re.search(p, text)]
def detect_sentiment(text):
r = client.chat.completions.create(
model="gpt-4o-mini", temperature=0, max_tokens=5,
messages=[{'role':'user','content':
f'Classify sentiment in ONE word: angry|frustrated|neutral|positive. Message: "{text}"'}])
return r.choices[0].message.content.strip().lower()
class InsureBankChatbot:
def __init__(self):
self.history = []
self.turn = 0
def chat(self, msg):
self.turn += 1
print(f'\nTurn {self.turn}')
print(f' User: {msg}')
# Layer 1: PII Guard
pii = detect_pii(msg)
if pii:
reply = f'Do not share {pii} here. Visit insurebank.in/secure.'
print(f' [PII Guard: {pii}]\n Zara: {reply}'); return reply
# Layer 2: Topic Guard
if any(kw in msg.lower() for kw in OFF_TOPIC):
reply = 'I am Zara, InsureBank assistant. I can only help with loan and insurance queries.'
print(f' [Topic Guard]\n Zara: {reply}'); return reply
# Layer 3: Sentiment
sentiment = detect_sentiment(msg)
print(f' [Sentiment: {sentiment}]')
prefix = ''
if sentiment in ('angry','frustrated'):
prefix = 'I completely understand your frustration. Let me help. '
# Layer 4: LLM with system prompt
self.history.append({'role':'user','content':msg})
msgs = [{'role':'system','content':SYSTEM_PROMPT}] + self.history
r = client.chat.completions.create(
model="gpt-4o-mini", temperature=0.3, max_tokens=400, messages=msgs)
reply = r.choices[0].message.content.strip()
final = prefix + reply
self.history.append({'role':'assistant','content':final})
if len(self.history) > 20: self.history = self.history[-20:]
```

```
print(f' Zara: {final}'); return final
def reset(self):
self.history = []; self.turn = 0
print('\n[Session reset]')
def demo():
bot = InsureBankChatbot()
print('=== DEMO SCENARIOS ===')
bot.chat('What is the minimum credit score for a personal loan?')
bot.chat('And the maximum loan amount?')
bot.reset()
bot.chat('What is the current Bitcoin price?')
bot.reset()
bot.chat('My Aadhaar is 9876 5432 1012. Check my eligibility.')
bot.reset()
bot.chat('This is ridiculous! I applied 2 weeks ago and nobody called!')
bot.reset()
bot.chat('Is your home loan rate better than HDFC Bank?')
def interactive():
bot = InsureBankChatbot()
print('=== InsureBank Chatbot — Interactive Mode ===')
print("Zara: Hello! I'm Zara from InsureBank. How can I help you?")
while True:
try: msg = input('\nYou: ').strip()
except (EOFError, KeyboardInterrupt): break
if not msg: continue
if msg.lower() == 'quit': break
if msg.lower() == 'reset': bot.reset(); continue
bot.chat(msg)
if __name__ == '__main__':
import sys
if len(sys.argv) > 1 and sys.argv[1] == '--interactive':
interactive()
else:
demo()
```

7. Challenge Exercises

These challenges extend what you built in each lab. Try them in order — each one deepens a core prompt engineering concept.

Challenge 1
LAB 1

Add Step 5 — Save Output to JSON File

Extend run_pipeline() with a Step 5 that saves the complete result (applicant + risk + decision + letter) as a timestamped JSON file.

```
# Hint — Add Step 5 — Save Output to JSON File
import datetime
filename = f"InsureBank_{applicant['full_name'].replace(' ','_')}"
f"_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
with open(filename, 'w') as f:
json.dump({'applicant':applicant,'risk':risk,
'decision':decision,'letter':letter}, f, indent=2)
print(f'Saved to {filename}')
```

Challenge 2
LAB 2

Add Test 5 — Statistics Hallucination

Ask 'What was InsureBank's claim settlement ratio in 2023?' without data. It will hallucinate a percentage. Fix it using the uncertainty enforcement strategy.

```
# Hint — Add Test 5 — Statistics Hallucination
# Broken — model invents a percentage:
"What was InsureBank claim settlement ratio in 2023?"
# Fixed — add to prompt:
"If you do not have verified InsureBank statistics, say:
I don't have that data. Check insurebank.in/reports.
Never guess percentages, ratios, or customer counts."
```

Challenge 3
LAB 3

Add Business Logic to Validator

Extend validate() to check: if decision is REJECTED, approved_amount_inr must be 0. Then write a test that intentionally breaks this and confirms your validator catches it.

```
# Hint — Add Business Logic to Validator
# Add to validate() function:
if output.get('decision') == 'REJECTED':
if output.get('approved_amount_inr', 0) != 0:
errors.append('REJECTED must have approved_amount_inr=0')
# Test it — this should trigger the error:
bad = {'decision':'REJECTED','approved_amount_inr':50000,...}
print(validate(bad)) # Should show the business logic error
```

Challenge 4
LAB 4

Add Rate Limiter Guard

After 5 messages in a session, Zara should stop responding and direct the user to call 1800-INS-BANK.

```
# Hint — Add Rate Limiter Guard
# In chat() method — add at the top:
if self.turn > 5:
return ('You have reached the chat session limit. '
'Please call 1800-INS-BANK for further assistance.')
```

```
# Test it: run 6 turns and check the 6th response
```

Challenge 5
ALL LABS

Wire Lab 1 Pipeline into the Chatbot

When a user says 'I want to apply for a loan', the chatbot collects details over 3 conversational turns, then runs the full 4-step pipeline and returns the decision letter as Zara's reply.

```
# Hint — Wire Lab 1 Pipeline into the Chatbot
# Add state to InsureBankChatbot.__init__:
self.loan_state = None
self.loan_inputs = []
# In chat() — detect intent and collect:
if 'apply' in msg.lower() and self.loan_state is None:
self.loan_state = 'collecting'
return 'Great! Tell me your name, income, and credit score.'
elif self.loan_state == 'collecting':
self.loan_inputs.append(msg)
if len(self.loan_inputs) >= 2:
result = run_pipeline(' '.join(self.loan_inputs))
self.loan_state = None
return result['letter']
return 'Got it! What loan amount do you need and for what purpose?'
```

8. Key Concepts Reference

All 17 prompt engineering techniques used across the 4 labs, mapped to their insurance application and the reason they work.

Lab	Technique	Insurance Application	Why It Works
LAB 1 Step 1	Zero-Shot JSON Extraction	Parse applicant text → structured data	Explicit schema + format constraint removes ambiguity
LAB 1 Step 2	Chain-of-Thought (CoT)	Multi-factor credit risk scoring	Step-by-step reasoning improves accuracy on complex analysis
LAB 1 Step 3	Policy Grounding	Loan decision against binding rules	Model limited to what you inject — cannot fabricate criteria
LAB 1 Step 4	Tone + Persona Control	APPROVED vs REJECTED letters	Named emotional register produces predictable and consistent tone
LAB 2 Test 1	Context Injection	Prevent policy hallucination	Closed-world: model only knows what you tell it
LAB 2 Test 2	Uncertainty Enforcement	Prevent fake IRDAI citations	Saying 'I don't know' is safer than inventing sources
LAB 2 Test 3	Catalog Grounding	Prevent phantom product invention	Product catalog as single source of truth for existence
LAB 2 Test 4	Formula Injection	Accurate EMI calculation	Exact formula leaves zero room for approximation
LAB 3 Fix 1	Two-Prompt Validator-Fixer	Repair wrong field names	Separates generation from correction — clear failure mode
LAB 3 Fix 2	Robust JSON Parsing	Handle mixed text + JSON output	Fence stripping + start-index search handles all model formats
LAB 3 Fix 3	Policy Review Pass	Catch logic-violating decisions	Second-opinion prompt as automated QA layer
LAB 3 Fix 4	Retry-with-Feedback Loop	Production output resilience	Validation errors become self-correction context
LAB 4	System Prompt Governance	Chatbot rules + persona + constraints	System message applied to every turn — immutable rule layer
LAB 4	PII Pre-Filter (Regex)	Block sensitive data before LLM	Intercept before API call — faster, cheaper, compliant
LAB 4	Topic Boundary Guard	Restrict to insurance domain only	Keyword check + system rule creates hard domain boundary
LAB 4	Sentiment Classification	Detect anger → escalation path	Micro-LLM call (5 tokens) routes user to correct response branch
LAB 4	Multi-Turn Message History	Conversational memory across turns	LLM is stateless — full history sent every call maintains context

■ GOLDEN RULE OF PROMPT ENGINEERING: The more specific and constrained your prompt, the more reliable the output. Vague prompts produce vague results. Production-grade prompts read like legal contracts — explicit, exhaustive, unambiguous.

Run All Labs — In Order

```
bash — Run All Labs
# ■■ Complete Execution Sequence ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Install dependencies (once)
```

```
pip install openai python-dotenv
# LAB 1 — Multi-step loan eligibility workflow
python app/loan_eligibility.py
# LAB 2 — Hallucination testing and grounding
python app/hallucination_test.py
# LAB 3 — Fix incorrect outputs
python app/fix_bad_outputs.py
# LAB 4 — Guard chatbot (automated demo scenarios)
python chatbot/guard_chatbot.py
# LAB 4 — Guard chatbot (interactive — chat with Zara yourself!)
python chatbot/guard_chatbot.py --interactive
```